

Troubleshooting e FAQ

Erros comuns e diagnóstico

Ciclartech · Plataforma de Gestão de Ativos Assistivos

Troubleshooting e FAQ

Erros comuns e como diagnosticar, organizados por sintoma. Se o problema não estiver aqui, comece pelo painel de logs da Vercel (produção) ou pelo console do `runserver` (local) — ver `docs/GUIA_OPERACOES.md` §9.

Setup / desenvolvimento local

`ImproperlyConfigured`: Configuração de produção insegura

A aplicação se recusa a subir porque `DJANGO_DEBUG=False` (ou ausente, com default de produção) e a `SECRET_KEY` / `ALLOWED_HOSTS` estão inseguras. Comportamento intencional (`ciclartech/seguranca.py`), não um bug. Corrigir:

- Definir `DJANGO_SECRET_KEY` com uma chave gerada (nunca a de `.env.example`): `bash python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`
- Definir `DJANGO_ALLOWED_HOSTS` sem `*`.
- Em desenvolvimento local, garantir `DJANGO_DEBUG=True` no `.env` — nesse modo a validação não é bloqueante.

`django.db.utils.OperationalError`: could not connect to server

Postgres local não está rodando, ou `DATABASE_URL` no `.env` aponta para host/porta/credencial errados. Se estiver usando `docker compose`, confirmar que o serviço `db` subiu (`docker compose ps`); se for Postgres nativo, confirmar usuário/banco criados (`createuser ciclartech --pwprompt --createdb / createdb -0 ciclartech ciclartech`, conforme `README.md`).

Tela em branco / listagem vazia mesmo com dados no banco

Sintoma clássico de estar fora do contexto de tenant — não é bug de consulta, é o isolamento fail-closed funcionando (`core.tenancy`, ver `docs/GUIA_DESENVOLVEDOR.md` §4). Checar:

- Está logado com um usuário que tem `tenant` atribuído (não um Owner acessando `/app/`, que é área de cliente)?
- Se é um script/shell/management command, o `ContextVar` do tenant corrente foi setado (`core.tenancy.set_current_tenant_id`)? Fora de uma requisição HTTP passada pelo `TenantMiddleware`, ele não é populado sozinho.

`python manage.py test` **falha em massa depois de alterar** `models.py`

Provavelmente falta migration. Gerar e conferir antes de rodar de novo:

```
python manage.py makemigrations
python manage.py migrate
```

Revisar o arquivo de migration gerado antes de commitar — em especial `AddField` sem `default / null` em tabela já populada, que trava a migration em produção.

Login e autenticação

"Conta bloqueada temporariamente" mesmo com senha correta

Bloqueio por tentativas (`contas/bloqueio.py`) — 5 tentativas por identificação de usuário ou 20 por IP numa janela de 15 minutos (`SEGURANCA_LOGIN_MAX_TENTATIVAS_IDENTIFICACAO / _IP` , `SEGURANCA_LOGIN_JANELA_MINUTOS`). Esperar a janela expirar, ou (suporte) confirmar no registro de auditoria (`bloqueio_tentativas`) se foi engano de digitação repetido ou tentativa de ataque. Não há hoje um botão de "desbloquear na marra" — é temporal por desenho.

Em produção atrás de proxy (Vercel), se **todo mundo** de uma instituição está sendo bloqueado junto por causa de poucos usuários errando a senha, verificar

`DJANGO_PROXIES_CONFIAVEIS` — se não estiver configurado corretamente, o IP registrado é o da borda da Vercel (o mesmo para todo mundo), não o do cliente, e o limite por IP passa a valer para a instituição inteira, não por usuário real.

Link de redefinição de senha não chega

Sem `DJANGO_EMAIL_BACKEND` configurado para SMTP, o link só é escrito no log (comportamento padrão, não falha silenciosa disfarçada — ver `ciclartech/settings.py` `§"E-mail"`). Ver `docs/GUIA_OPERACOES.md §10` para diagnóstico e alternativa (Admin gera nova senha manualmente).

"Usuário de cliente precisa estar vinculado a um tenant" ao criar usuário via shell/Admin

`Usuario.clean()` exige `tenant_id` para qualquer usuário que não seja `is_platform_staff` nem `is_superuser` . Ao criar usuário fora das telas padrão (shell, fixture, script), sempre passar `tenant=` .

Erro 403 / 404 inesperado

404 numa ficha/documento que "deveria existir"

Quase sempre é o escopo por tenant **ou** por unidade funcionando como projetado, não um bug de dado perdido. Confirmar:

1. O registro pertence ao mesmo `tenant` do usuário logado?

2. Se o usuário é Gestor/Funcionário (não Admin), o registro pertence a uma `Unidade` atribuída a ele (`Usuario.unidades`)? Admin sempre vê tudo do tenant; Gestor/Funcionário só o que está na lista de unidades deles.
3. Beneficiário/Ativo **sem** unidade atribuída fica visível a todo o tenant (regra de "sem unidade = visível à organização") — se o registro deveria aparecer para todos e não aparece, confirmar que `unidade` está mesmo vazio e não apontando para outra unidade por engano.

Ver `core/unidades.py` para a lógica exata, e `docs/GUIA_DESENVOLVEDOR.md` §5.

403 "Esta operação exige confirmação (POST)"

Ações que mudam estado visível para outras pessoas (alternar módulo, suspender tenant, anonimizar titular) exigem POST por desenho — GET nessas rotas sempre devolve 403. Não é erro de permissão de papel, é proteção contra CSRF/engano de clique em link. Verificar que o botão do template realmente submete um form POST, não é um `<a href>`.

403 ao tentar gerenciar outro usuário

`Usuario.pode_gerenciar()` : Admin gerencia Gestor/Funcionário; Gestor não gerencia Admin nem outro Gestor de nível igual/maior; ninguém gerencia usuário de outro tenant. Confirmar `nivel_hierarquico` dos dois papéis envolvidos (`contas_papel`) se o comportamento parecer errado.

Upload de arquivo

"Tipo de arquivo não permitido" com um arquivo que parece válido

`core/validadores.py::validar_upload / validar_upload_imagem` checam extensão e magic number do conteúdo, não só a extensão do nome do arquivo — um arquivo renomeado (ex.: SVG com script, renomeado para `.png`) é rejeitado mesmo com extensão válida. Se o arquivo é legítimo e está sendo rejeitado, confirmar que o conteúdo real bate com o tipo declarado (reexportar/reconverter o arquivo costuma resolver arquivo corrompido ou com metadado inconsistente).

Upload funciona local mas some em produção (Vercel)

A Vercel não tem disco persistente entre execuções — se `STORAGES` caiu para `FileSystemStorage` em produção (variáveis `DJANGO_STORAGE_S3_ACCESS_KEY_ID / _SECRET_ACCESS_KEY` ausentes ou erradas), todo upload grava num disco que desaparece na próxima invocação. Confirmar as variáveis de Storage S3/Supabase no painel da Vercel — ver `README.md` §"Storage de mídia".

Documento de beneficiário não abre / baixa como binário estranho

Comportamento esperado: `baixar_documento` sempre entrega como `attachment` com `Content-Type: application/octet-stream`, nunca renderiza inline — proteção contra XSS via SVG/HTML disfarçado de imagem (ver `beneficiarios/tests/test_lgpd.py::DocumentoProtegidoTest`). O navegador deve oferecer "salvar como"; abrir depois de baixado normalmente.

Notificações

Notificação não foi "enviada" de verdade (WhatsApp/e-mail)

Nesta fase o backend de notificação **registra e "envia" via log estruturado** — não há integração real com WhatsApp Business API/SMTP para este canal (ver `notificacoes/services.py::_despachar`). Ver o registro em `notificacoes_notificacaoenviada` /no log para confirmar que o sistema processou corretamente; a entrega real ao destinatário é ponto de extensão pendente, não falha do sistema atual.

Job diário não disparou nenhuma notificação

Conferir, nesta ordem: (1) existe `DetalheEmprestimo` com `data_prevista_devolucao` na janela esperada (7 dias/hoje/atrasado)? (2) o `NotificacaoTemplate` do tipo correspondente existe para o tenant (`UniqueConstraint (tenant, tipo)`) — sem template, não há o que renderizar; (3) em produção, o cron da Vercel de fato disparou — ver `docs/GUIA_OPERACOES.md` §5 para confirmar `CRON_SECRET` e o log do endpoint `/cron/notificacoes-diarias/`.

Máquina de estados / movimentação de ativo

"Transição inválida" ao tentar uma ação que parecia óbvia

A tabela de transições é fechada por desenho (`ativos/domain/state_machine.py`) — uma ação só é permitida a partir de estados explicitamente listados. Casos que frequentemente surpreendem:

- Não dá para **transferir** um ativo **emprestado** entre unidades — ele está fisicamente com o beneficiário; primeiro precisa devolver.
- Devolução (`DEVOLUCAO`) exige um destino explícito (`disponivel / higienizacao / manutencao`) — não tem "devolução simples" sem decidir o destino.
- Um ativo `baixado` é estado terminal — não há transição de volta.

Ver a tabela completa em `docs/FLUXOS_DE_NEGOCIO.md` §1 ou diretamente em `ativos/domain/state_machine.py` (fonte de verdade).

Erro ao tentar apagar uma `Movimentacao` ou `RegistroAuditoria`

Comportamento esperado, não bug: os dois models bloqueiam `delete()` (e `RegistroAuditoria` também bloqueia `save()` de `update`) — são históricos append-only por desenho de auditoria/negócio. Se o dado realmente precisa sumir por decisão de LGPD, o caminho correto para `Beneficiario` é a anonimização (`beneficiarios/lgpd.py::anonimizar`), que preserva o histórico patrimonial mas remove identificação — nunca excluir a movimentação em si.

Deploy (Vercel)

400 Bad Request logo após um deploy novo

Provável `Host` não reconhecido — a Vercel gera um domínio novo por deploy (`VERCEL_URL`) além do domínio estável (`VERCEL_PROJECT_PRODUCTION_URL`); `settings.py` já registra os dois automaticamente em `ALLOWED_HOSTS` . Se estiver usando domínio próprio customizado, ele precisa estar explicitamente em `DJANGO_ALLOWED_HOSTS` .

Loop infinito de redirecionamento HTTPS

Sintoma de `SECURE_PROXY_SSL_HEADER` não reconhecendo o cabeçalho que a Vercel envia — já configurado em `settings.py` para (`"HTTP_X_FORWARDED_PROTO", "https"`), mas confirmar se não há um proxy adicional na frente (CDN customizado) que não repassa esse cabeçalho.

CSS/imagem de foto de ativo bloqueado no navegador (CSP)

Se `MEDIA_STORAGE_HOST` não corresponde ao host real do endpoint S3 configurado, o `img-src` da CSP não libera o domínio do storage e o navegador bloqueia a imagem. Conferir `DJANGO_STORAGE_S3_ENDPOINT_URL` e `core/middleware.py::_diretiva_img_src` .

Onde olhar quando nada disso resolve

1. Reproduzir localmente com `DJANGO_DEBUG=True` para ver o traceback completo (produção nunca expõe stack trace ao usuário).
2. Rodar a suíte de testes do app envolvido — `python manage.py test <app>` — para confirmar se é regressão conhecida.
3. Verificar se o comportamento tem teste cobrindo o cenário esperado (`grep` pelo nome do model/view em `*/tests/`) — muitas vezes o "por que funciona assim" está documentado na docstring do teste.
4. Consultar `docs/business-rules/<assunto>.md` — se o comportamento é deliberado, geralmente está justificado ali.